Lab 6 Inode identity and recursive pathname search

Tacu Igor, FAF 242
Operating Systems, Technical University of Moldova
New xv6 reproduction on 9 November 2026

Scope and implementation

This experiment examines two different tasks: reading metadata for a supplied name and walking a directory tree to discover names. The complete implementations are included as `xv6/user/mystat.c` and `xv6/user/find.c`, together with their UPROGS entries and the full kernel tree needed to build them. The following results were collected from that tree in QEMU with one emulated hart, using a fresh generated filesystem image.

The tests use a disposable file named `specimen`; they do not remove the xv6 README. This preserves the teaching image while still testing what happens when a name is unlinked. The test tree has both files and directories whose basenames can match a search target, so successful recursion alone cannot hide a failure to report directories.

Reading metadata without confusing a name with an inode

`mystat` loops over its operands and calls `stat` for each one. It translates the three public xv6 types into file, directory, and device labels, and prints the inode number, link count, and byte size. If one lookup fails, it records failure and continues through later operands instead of aborting at the first bad name. The size uses the format for a 64-bit unsigned value rather than treating the field as a small integer.

The initial checks identified the root as directory inode 1 and console as a device. Writing `igor-file` plus its newline produced a ten-byte regular file at inode 33 in this image. Creating `alias` with `ln` made both names report the same inode and link count two. Removing only `specimen` left `alias` with count one and the same ten bytes, and `cat alias` still read the text. These observations separate three properties that are easy to conflate: the pathname used for lookup, the inode that holds identity and metadata, and the data still reachable through another name.

```
$ echo igor-file > specimen
$ mystat specimen
specimen: type=file inode=33 links=1 bytes=10
$ ln specimen alias
$ mystat specimen alias
specimen: type=file inode=33 links=2 bytes=10
alias: type=file inode=33 links=2 bytes=10
$ rm specimen
$ mystat alias
alias: type=file inode=33 links=1 bytes=10
$ cat alias
igor-file
$ mystat missing-path
mystat: cannot stat missing-path
```

Inode numbers depend on allocation order. The tested invariant is that both names identify one inode and that removing one name preserves the remaining link and contents.

A find implementation that also reports directories

The search utility takes a starting pathname and a basename. After stat it compares the current name before deciding whether to recurse, allowing directories themselves to match. It copies the

fixed-width `dirent.name` into a DIRSIZ-plus-one buffer and adds a terminator rather than assuming the kernel entry is already a C string.

The loop skips unused entries, dot, and dot-dot. Before constructing a child path, it checks the parent, separator, name, and terminator against its 512-byte buffer. Lookup and read failures produce diagnostics and a failure status. Every opened directory is closed after traversal. Ancestor descriptors remain open during recursion, so very deep trees are limited by xv6's descriptor limit.

Tests that distinguish traversal from correct matching

The tree contains `tree/target` and `tree/branch/target`, plus a directory `tree/branch/targetdir`. Searching for `target` printed the two files; searching for `targetdir` printed the directory; searching for `branch` printed the intermediate directory. A fourteen-character filename checked the DIRSIZ boundary, and a missing starting path exercised the diagnostic path. Invoking `find` without the two operands printed usage rather than interpreting uninitialized arguments.

```
$ mkdir tree
$ mkdir tree/branch
$ echo root-copy > tree/target
$ echo nested-copy > tree/branch/target
$ mkdir tree/branch/targetdir
$ find tree target
tree/branch/target
tree/target
$ find tree targetdir
tree/branch/targetdir
$ find tree branch
tree/branch
$ echo edge > tree/abcdefghijklmn
$ find tree abcdefghijklmn
tree/abcdefghijklmn
$ find missing-path target
find: cannot stat missing-path
$ find
usage: find start basename
$
```

These tests would expose a common bug in which basename comparison is performed only for regular files. Such a version can descend into `branch` and still fail to print `branch` or `targetdir`. The directory-target assertions in `scripts/run-xv6.py` verify that behavior explicitly. The raw console output is retained in `Lab5-6/evidence/xv6.txt`, so the displayed paths can be checked against the commands that constructed them.

Why unlink does not always free the blocks immediately

In `kernel/sysfile.c`, `sys_unlink` removes the name and updates link counts. In `kernel/fs.c`, `iput` waits for zero links and the final relevant in-memory reference before `itrunc` frees data blocks. An open reference can therefore preserve an unlinked inode.

The experiment proves survival through another hard link. The open-but-unlinked explanation comes from source inspection; it is a separate lifetime case.

Running `scripts/run-xv6.py` rebuilds the image and checks these distinctive link and search results. A fresh generated disk prevents leftover test names from affecting a rerun.